# I Taught an Agent to Build TV Apps

Lessons from building a coding harness

```
$ tv-build claude-run my-inputs --detach --yes
✓ run detached · runId 7f3a…  status: running
```

WHOAMI

# Giovanni Laquidara

## Sr. Developer Advocate & Builder

Amazon · London

giolaq.dev · github.com/giolaq · @giolaq

Agentic AI · Devices · Cross-platform · Harness engineering

THE PROBLEM

# TV App development is HARD

### Fragmentation

Many OSes · Many toolchains · Many guidelines

### Control

Remote and Focus Management

### Performance

Hardware with very limited resources

THE PROBLEM · FRAGMENTATION

# More than 5 different platforms







**Toolchains**

Gradle · Xcode · Expo · Vega SDK · bundlers

**Constraints**

Focus engines · key codes

**Operations**

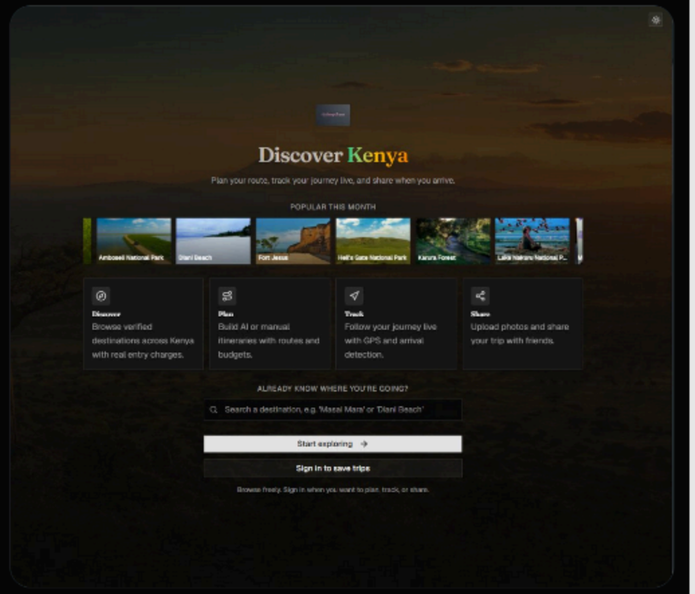
Generating code ≠ *operating* a platform

THE HYPE

# AI Is Good at One-Shotting...

**myk.ih_1** @ih_myk56735 · 9h

GPT 5.6 just one-shotted me a full stack web app in literally 40 minutes. work that would take me **one** day using other models. GPT 5.6 is such a leap.

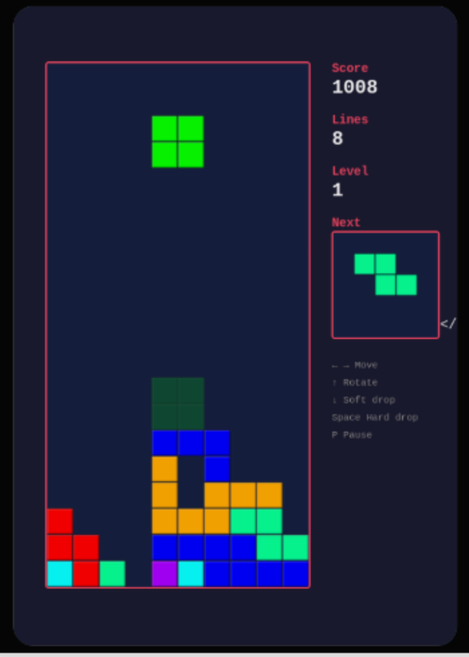



1 · 6

**aljaz** @aaaljaz · 21h

**one shotted** completely working tetris game with Qwopus3.6-27B-v2-MTP-GGUF Q3_K_M on mobile A5000 (16gb ram).

took some tuning of llama.cpp parameters and testing to figure out whats the best setup but then it cooks (also literally since thermals on laptops are horrible 💩)




**red** 📌 @redkendl · 9h

🚨Kimi K3 just **one-shotted** this 3D paper plane game.

I'm genuinely shocked. This is Fable 5 / GPT-5.6-level output.

Chinese AI labs are not 8 months behind the frontier anymore. They're right there.


2 · 2 · 45 · 4.1K

THE REALITY

# ...but not TV apps

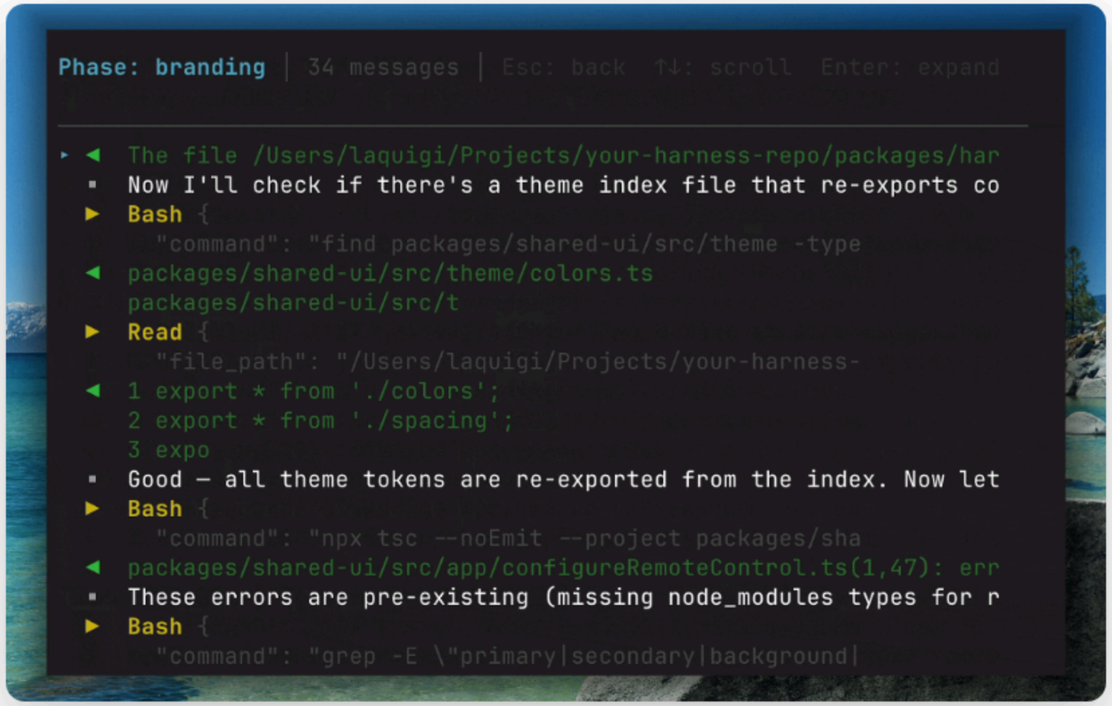
node ⌥⌘1

```
› create a tv app in React Native

● I'll create a React Native TV app. First, let me check the environment to pick the right
  toolchain — this machine has Amazon Vega (Kepler) tooling configured, so I want to see
  whether you have the Vega SDK installed or whether standard react-native-tvos is the
  better fit.

  Listed 1 directory (ctrl+o to expand)

● Bash(which kepler velocity vega 2>/dev/null; ls ~/.kepler 2>/dev/null | head; node
      --version; npm --version)
  └ /Users/laquigi/vega/bin/kepler
    velocity not found
    vega () {
    … +9 lines (ctrl+o to expand)

● Skill(amazon-devices-vega-build-and-run)
  └ Successfully loaded skill

● You have the Vega (Kepler) SDK installed, so I'll build a Vega React Native TV app. Let
  me pull the official docs for scaffolding and building.

  Called amazon-devices-buildertools-mcp 2 times (ctrl+o to expand)

● Now scaffolding the app with the Kepler CLI. Let me check available templates first.

● Bash(kepler project list-templates 2>&1)
  └ Available templates:

    helloWorld:        A basic React Native for Vega application, built with v0.72
    … +5 lines (ctrl+o to expand)
```

THE DECISION

# So I Taught an AI Agent How to Build a TV App

I actually built a harness.




**Observable**

**Configurable**

**Independent**

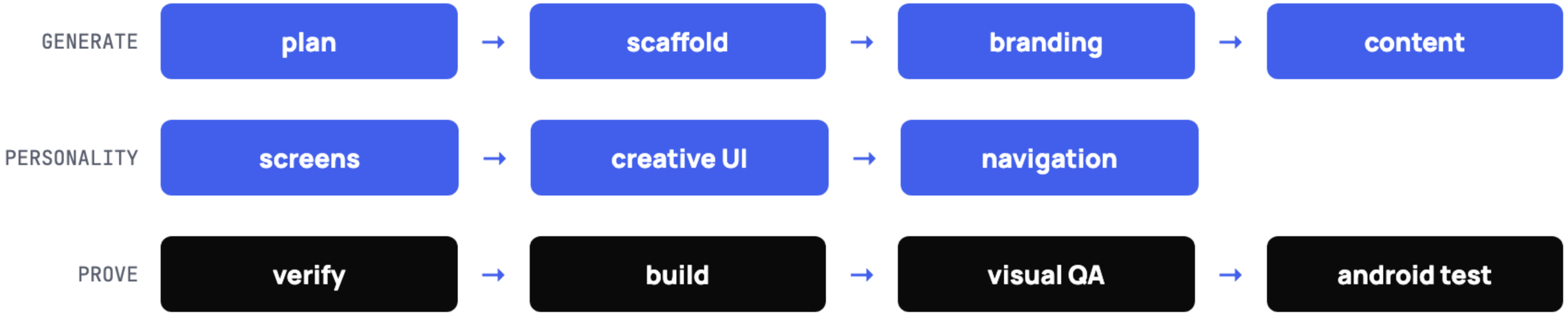
THE DECISION · MEET TV-BUILD

# TV Build: JSON + Text In, Working TV App Out

```
$ tv-build init cooking-shows
$ tv-build claude-run cooking-shows --detach --yes
✓ run detached · runId 7f3a…  status: running
```

THE HARNESS · THE PIPELINE

# One Pipeline, Many Phases



THE HARNESS · THE PHASES

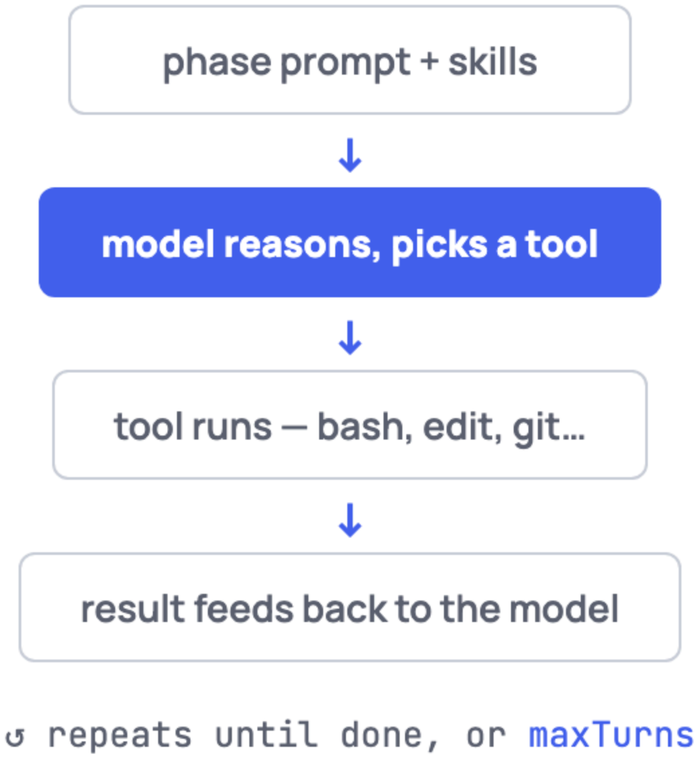
# What Each Phase Actually Does

| Phase | What happens |
| --- | --- |
| `plan` | Reads your inputs, decides screens and navigation structure |
| `scaffold → branding → content` | Clones the template, applies your colors, wires your data into hooks |
| `screens → creative_ui → navigation` | Customizes screens, adds typography and focus animations |
| `verify → build_loop` | TSC, focus checks, platform guards; web build and native prebuild |
| `visual_qa_loop → android_test_loop` | Screenshots every screen and grades them; D-pad testing on an emulator |

THE HARNESS · THE ENGINE

# Each Phase, One Agent

A fresh agent per phase



THE HARNESS · THE ENGINE

# Strands Agent SDK

```ts
import { Agent, tool, BeforeToolCallEvent } from '@strands-agents/sdk'
import z from 'zod'
import { writeFileSync } from 'fs'

const saveReport = tool({
  name: 'save_report',
  description: 'Save a research report.',
  inputSchema: z.object({
    title: z.string(),
    content: z.string(),
  }),
  callback: ({ title, content }) => {
    writeFileSync(`reports/${title}.md`, content)
    return `Saved ${title}.md`
  },
})

const agent = new Agent({ tools: [saveReport] })

agent.addHook(BeforeToolCallEvent, (event) => {
  const inp = String(event.toolUse.input)
  if (event.toolUse.name === 'save_report') {
    if (!inp.includes('[source]')) {
      event.cancel = 'Add source citations.'
    }
  }
})

await agent.invoke('Research AI agent frameworks')
```

STRANDS AGENTS SDK

Open-source agents SDK from AWS
Model + tools + prompt → agentic loop

strandsagents.com

THE HARNESS · THE EXECUTOR

# The Agent code

```
import { Agent } from "@strands-agents/sdk";

const tools = createStrandsTools({ appDir, workdir });
const model = createModel(phaseModels?.[phase] ?? modelConfig);

const agent = new Agent({ model, tools, systemPrompt,
                          plugins: [skillsPlugin] });

const stream = agent.stream(userMessage, { limits: { turns: maxTurns } });
let next = await stream.next();
while (!next.done) {
  handleStreamEvent(next.value);
  trackEventUsage(next.value);
  next = await stream.next();
}
```

THE HARNESS · THE TOOLS

# Tool code

```
const bashTool = tool({
  name: "bash",
  description: "Execute a shell command and return its output…",
  inputSchema: z.object({
    command: z.string(),
    cwd: z.string().optional(),
    timeout: z.number().optional(),   // default 60s
  }),
  callback: async ({ command, cwd, timeout }) => {
    try { return execSync(command, { cwd: execDir, timeout: timeout ?? 60_000 }); }
    catch (err) { return `Command failed (exit N)…`; } // errors as text, never thrown
  },
});
```

THE HARNESS · TOOL LAYERS

# Two Layers of Tools

### The agent holds

`bash` `read_file` `write_file` `edit_file` `list_files`
`git` `grep` `builder_tools`

8 Strands tools

### The harness holds

`android CLI` `adb` `gradle` `expo` `vega SDK`

The agent reaches platform CLIs through **bash + skills**

THE HARNESS · WHAT GOES IN

# content.json  What the App Shows

```
{
  "title": "My App",
  "categories": [
    { "id": "trending", "name": "Trending", "items": ["v1", "v2"] }
  ],
  "videos": [
    { "id": "v1", "title": "Some Video",
      "thumbnail_url": "https://…",
      "stream_url": "https://…", "stream_type": "hls" }
  ],
  "featured": ["v1"]
}
```

THE HARNESS · THE TASTE

# brand.json + prompt.txt  How It Should Feel

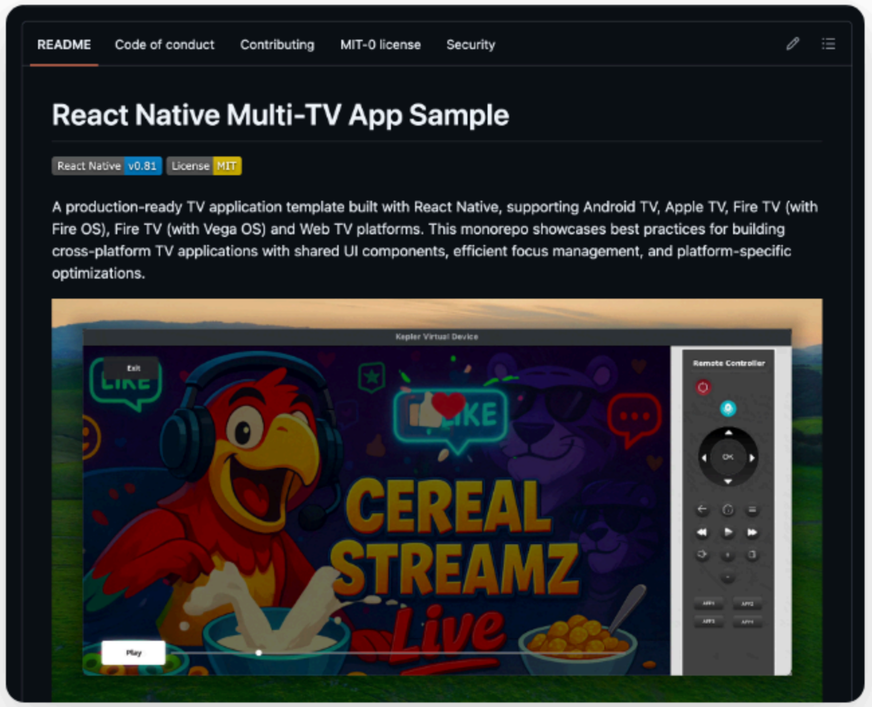
```
// brand.json — specific colors
{
  "name": "My App",
  "primary_color": "#6C5CE7",
  "accent_color": "#00CEC9",
  "background_color": "#0A0A12"
}
```

```
# prompt.txt — the vibe, in plain English
Dark and cinematic. Neon accent glows
on focus. Editorial typography.
The hero section should feel like
a movie premiere.
```

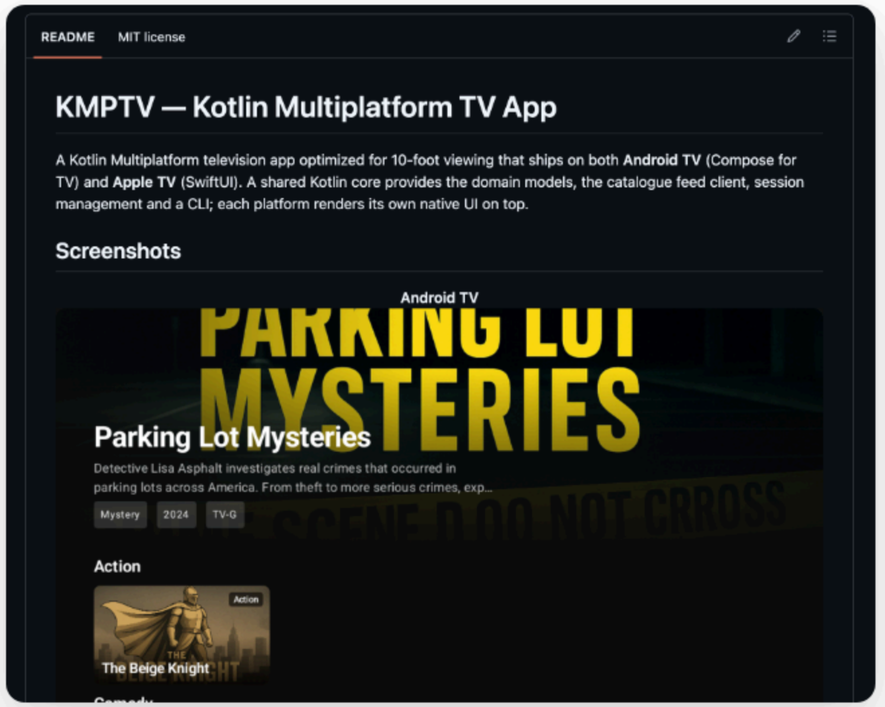
THE HARNESS · THE STRONG INPUT

# Start From a Working Template

The agent never invents from scratch. It customizes a proven, buildable template.




github.com/AmazonAppDev/react-native-multi-tv-app-sample




github.com/giolaq/kmptv

THE HARNESS · THE SKILLS

# Keep the Harness Thin and the Skills Specific

```
skills/
  rn-spatial-navigation/SKILL.md
  rn-theming/SKILL.md
  android-tv-testing/SKILL.md
  vega-sdk/SKILL.md
```

```
# frontmatter
name: rn-spatial-navigation
applies_to: [phase_screens, phase_static_check]
```

**“Thin harness, fat skills”**

**≈1,200 lines** TypeScript · **≈3,000 lines** markdown

THE HARNESS · A SKILL UP CLOSE

# What a Skill Actually Teaches

### Focus mechanics

Focus roots · D-pad events · overflow traps

### TV color physics

Panels over-saturate, desaturate or glow radioactive

### 10-foot judgment

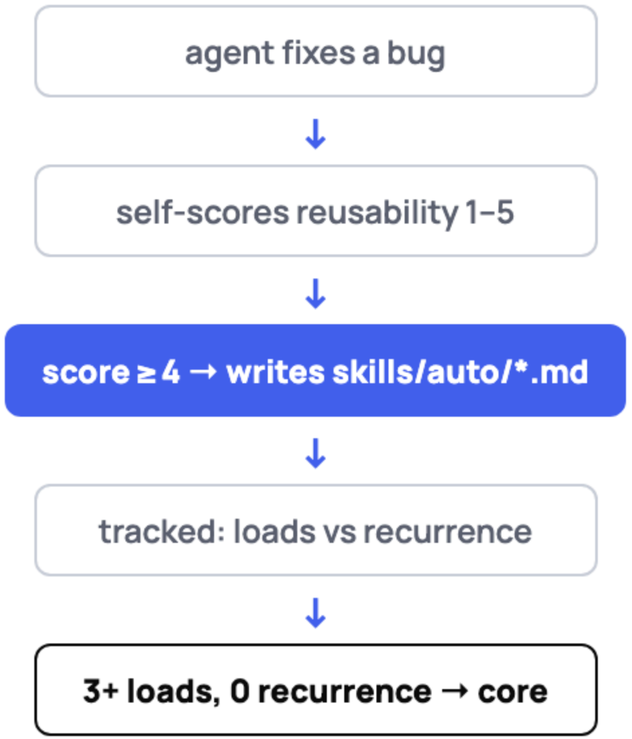
Cinematic scrims · items-per-rail · sequential clicks

Loaded per phase

THE HARNESS · SELF-IMPROVEMENT

# The Harness That Teaches Itself

Agent fixes bug → writes a skill.

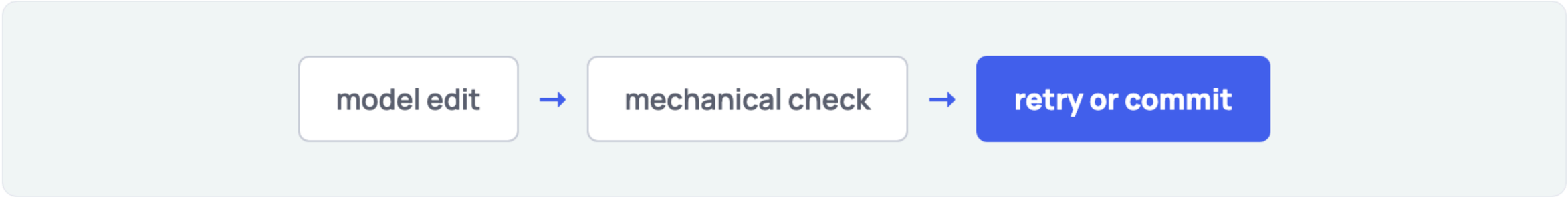



```
// baked into every phase prompt:
Rate the fix on reusability (1-5):
1 = only applies to this exact app
2 = might apply to similar apps
3 = applies to most TV apps with this nav style
4 = applies to ANY TV app using react-tv-space-navigation
5 = universal React Native TV pattern

Only skillify if score ⩾ 4.
Check for duplicates first: grep -rl in skills/auto/
```

**The agent scores itself** using the rubric in its own system prompt.

THE HARNESS · INSIDE EVERY PHASE

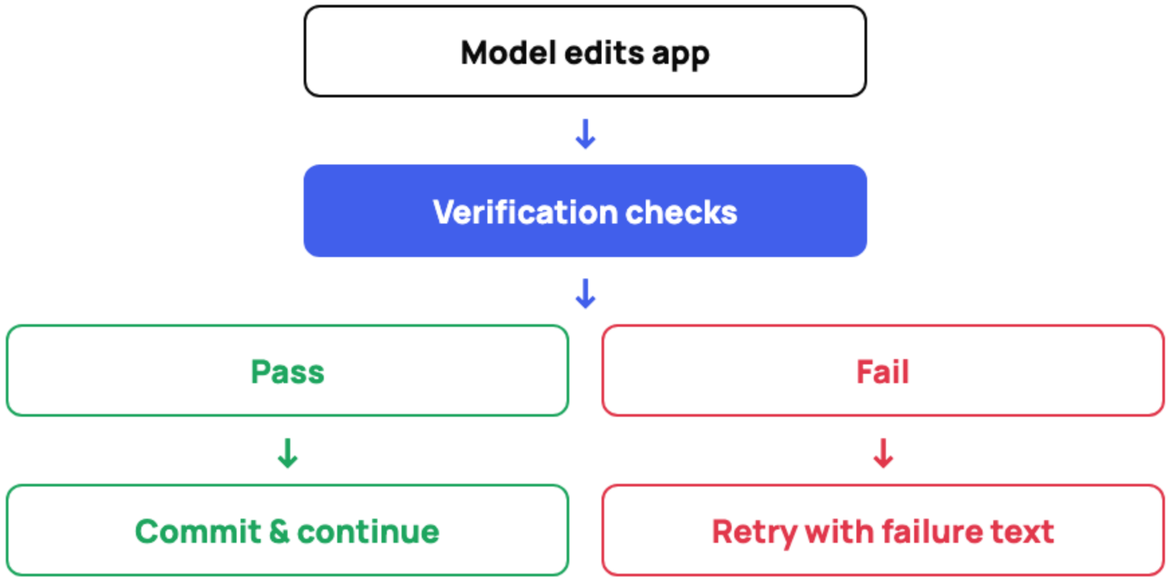
# The Same Loop Beats Inside Every Phase




Verification = **feedback** · git commit = **recovery point**

THE HARNESS · VERIFICATION

# Verification Is the Agent's Feedback Channel




### Mechanical checks

`file exists` `grep pattern` `TypeScript` `focus checks`
`build checks` `screenshots`

Cost cap exceeded → run **aborts**

ANDROID · THE STACK

# Three Android Tools

Auto-detect what's available. Use the best tool. Fall back gracefully.

`android CLI`

**Intelligence layer**

describe · emulator start · run · layout · screen capture · docs

`gradle`

**Build engine**

compileDebugKotlin · assembleDebug · installDebug (fallback)

`adb`

**Device wire**

devices · boot poll · **input dpad keyevent** · logcat · screencap (fallback)

`input dpad keyevent` not `input keyevent`. The dpad source matches how a physical remote sends events.

ANDROID · THE PREFERRED BACKEND

# Android CLI

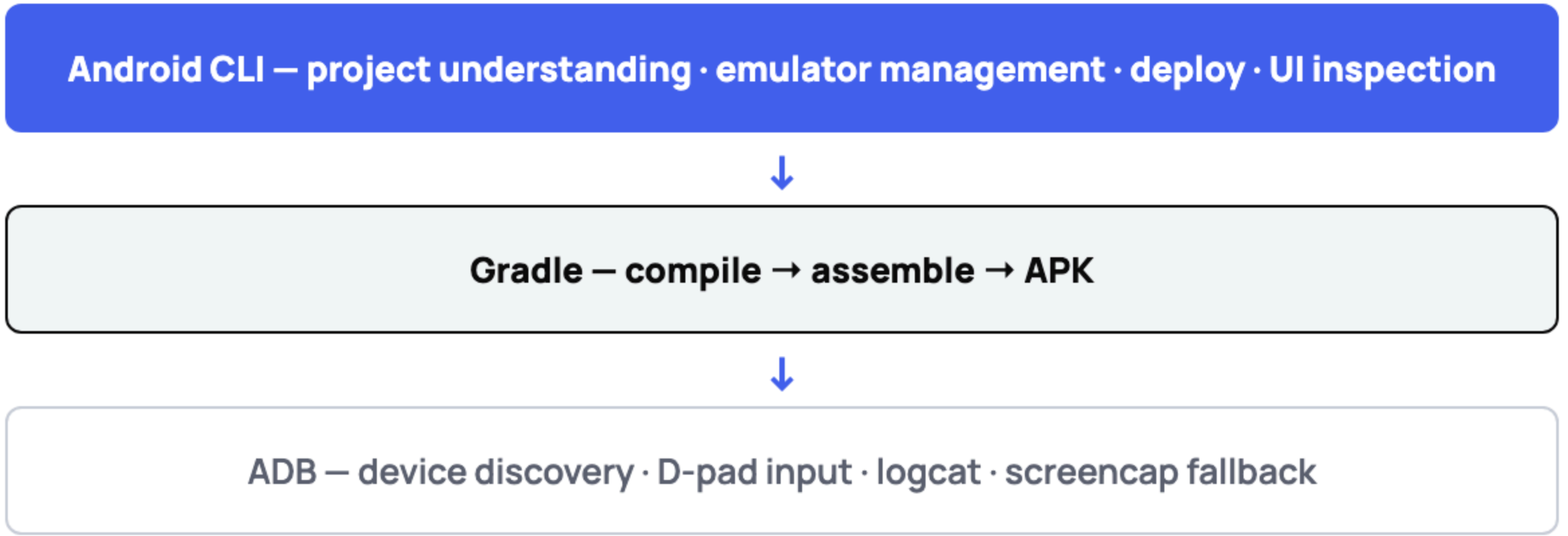
```
android info                  # detect SDK, environment
android init                  # install agent skill
android describe --project_dir=…  # project → APK path
android emulator list         # available profiles
android emulator start tv     # boot by profile name
android skills list --long    # loaded agent skills
android layout --pretty --output=…  # UI hierarchy → JSON
android screen capture --output=…   # screenshot → file
```

- ✓ Harness probes **android info** first
- ✓ Works → backend = `android-cli`
- ✓ Fails → falls back to `gradle-adb`
- ✓ `--require-android-cli` → **no silent fallback**

The agent never invents install commands or **guesses APK paths**.

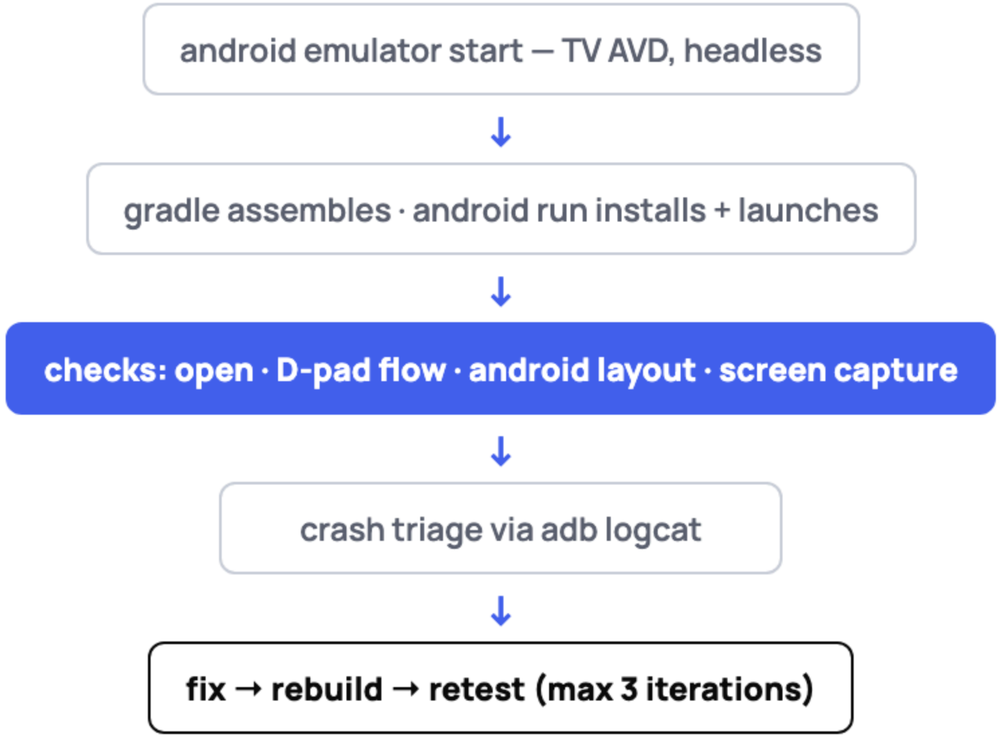
ANDROID · AUTO-DETECTION

# How They Layer Together



ANDROID · THE LAST PHASE

# android_test_loop: an Agent That Tests on a Real Emulator




**Bounded**

30 min · 3 iterations · `android-tv-testing` + `android-cli` skills

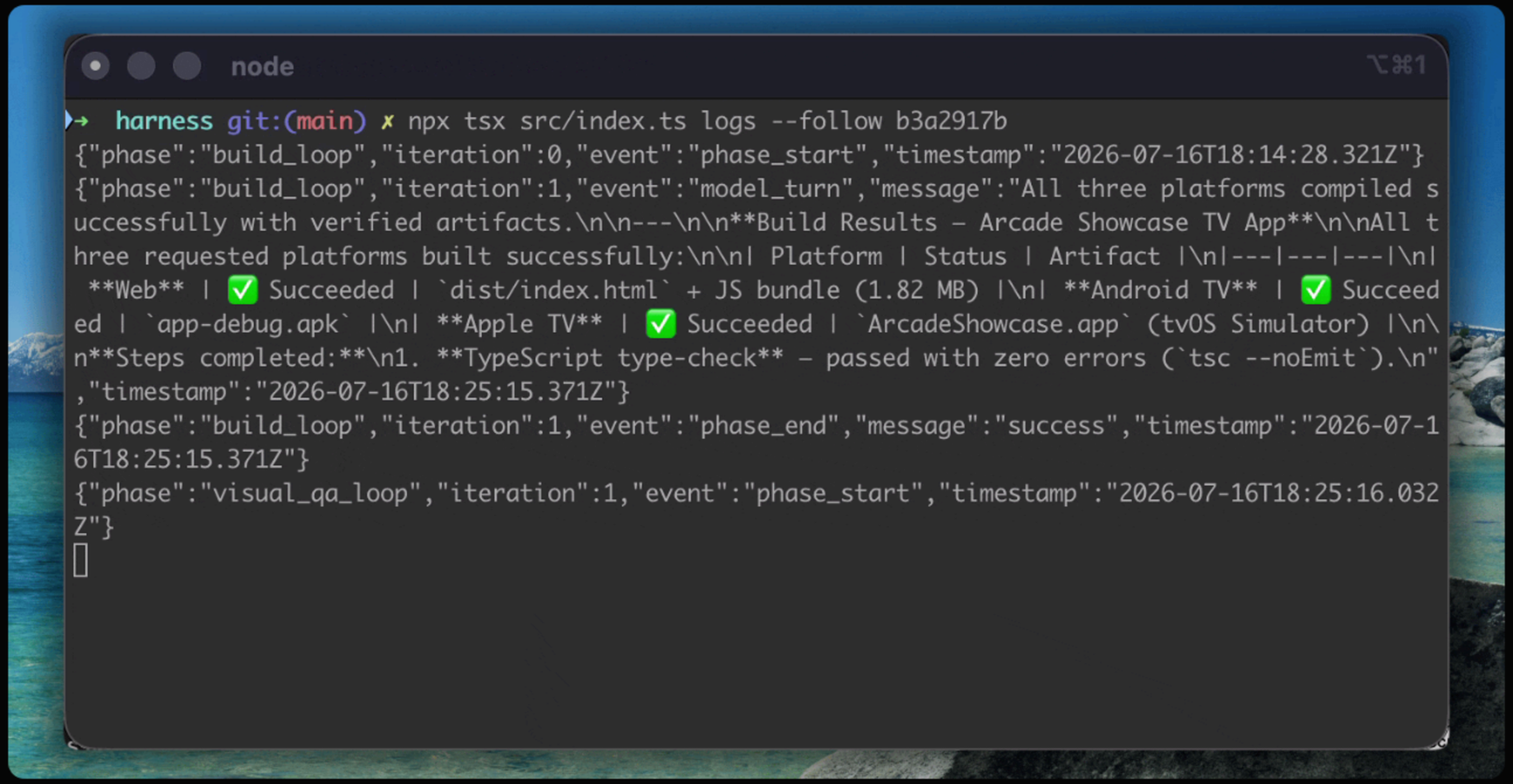
**Degrades honestly**

`tv-build doctor` → missing CLI? skipped, not failed.

OBSERVABILITY · LIVE

# Every Phase Narrates Itself

Human → TUI · agent → log · same events.



OBSERVABILITY · THE PAPER TRAIL

# Everything a Run Leaves Behind

THE HARNESS · THE CLI

# Commands

```
# generate
claude-run [dir]   # full pipeline (Claude CLI)
run [dir]          # full pipeline (API)
init <dir>         # scaffold inputs
validate [dir]     # check inputs
schema [name]      # describe input schema
# iterate
refine <t> "…"     # amend one phase
add-screen <N>    # add a screen
review [scope]     # TV-specific code review
```

```
# operate a detached run
status <runId>     # read run state
logs <runId>       # tail with --follow
abort <runId>      # stop it
replay <file>      # deterministic replay
# verify & environment
test-ui · visual-qa  # drive UI / grade screens
android <run>       # build, install, test on TV
doctor · vega-doctor  # check prerequisites
*-skills            # install/update/prune/consolidate
```

Every commands speaks `--json` and have `--detach`
Optimized for Agents

DEMO · RECOVERY

# Resumable

```
# pick up where it stopped
$ tv-build claude-run --resume
# re-run from a specific phase
$ tv-build claude-run --resume abc123 --from-phase verify
# just generate, skip build/QA
$ tv-build claude-run --generate-only
```

Failed run costs **one phase** and not all the previous runs.

ANY MODEL · THE SEAM

# Use your provider

```
// pipeline-engine.ts — the contract (all three engines satisfy it)
executor: (spec: PhaseSpec, attempt: number) ⇒ Promise<PhaseResult>;

// model-factory.ts — providers are a switch, not a rewrite
switch (config.provider) {
  case "bedrock":    return new BedrockModel({ modelId, region });
  case "anthropic":  return new AnthropicModel({ modelId });
  case "openrouter": return new OpenAIModel({ baseURL, fetch: patchedFetch });
}
```

ANY MODEL · TWO COMMANDS

# Use your model

```
$ tv-build claude-run my-inputs
```

**Recommended**

Claude CLI subprocess · stable · simple

```
$ tv-build run my-inputs
```

**Multi-provider**

Bedrock · OpenRouter · OpenAI · swap by config

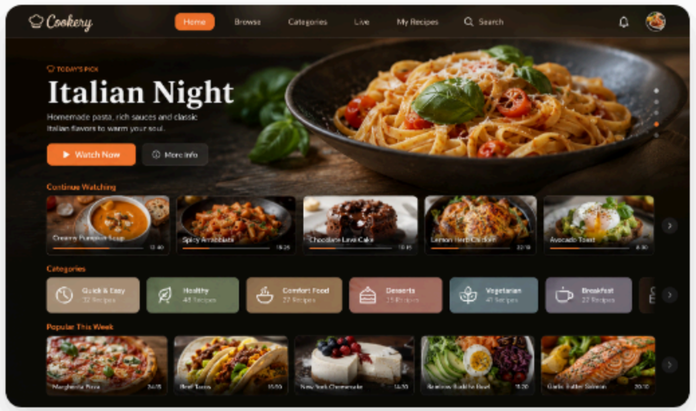
ANY MODEL · PER-PHASE CONFIG

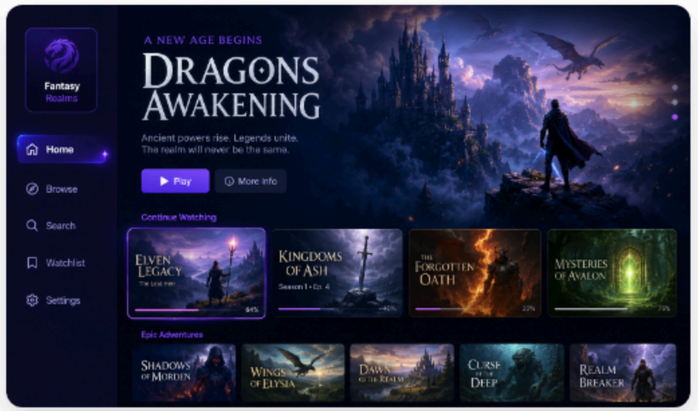
# Different Models for Different Phases

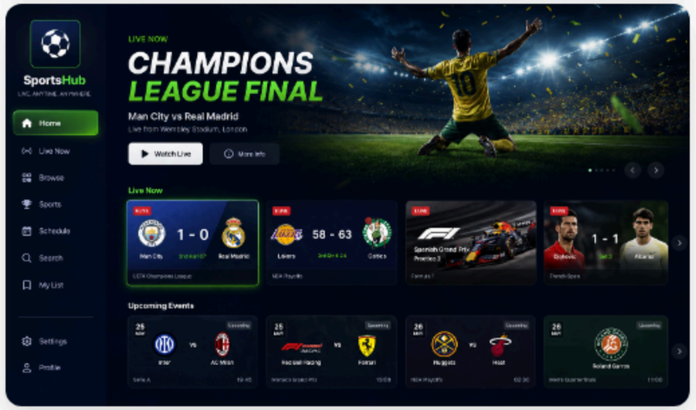
```
// harness.config.json — in the input dir, next to content.json
{
  "models": {
    "strandsProvider": {
      "provider": "openrouter",
      "modelId": "deepseek/deepseek-v4-flash"
    },
    "phaseModels": {
      "visual_qa_loop": {
        "provider": "openrouter",
        "modelId": "anthropic/claude-sonnet-4"
      }
    }
  }
}
```

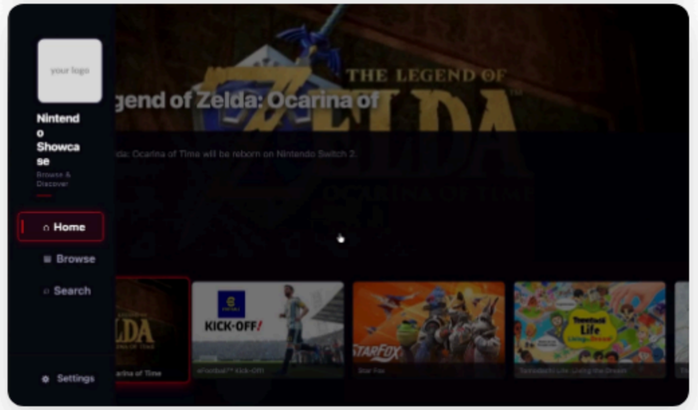
THE PROOF

# Examples



THE DECISION · RECEIPTS

# Eight Weeks, by the Numbers

**58**
unique generation runs

**467**
phase-level agent sessions recorded

**~30 min**
from JSON inputs to working app

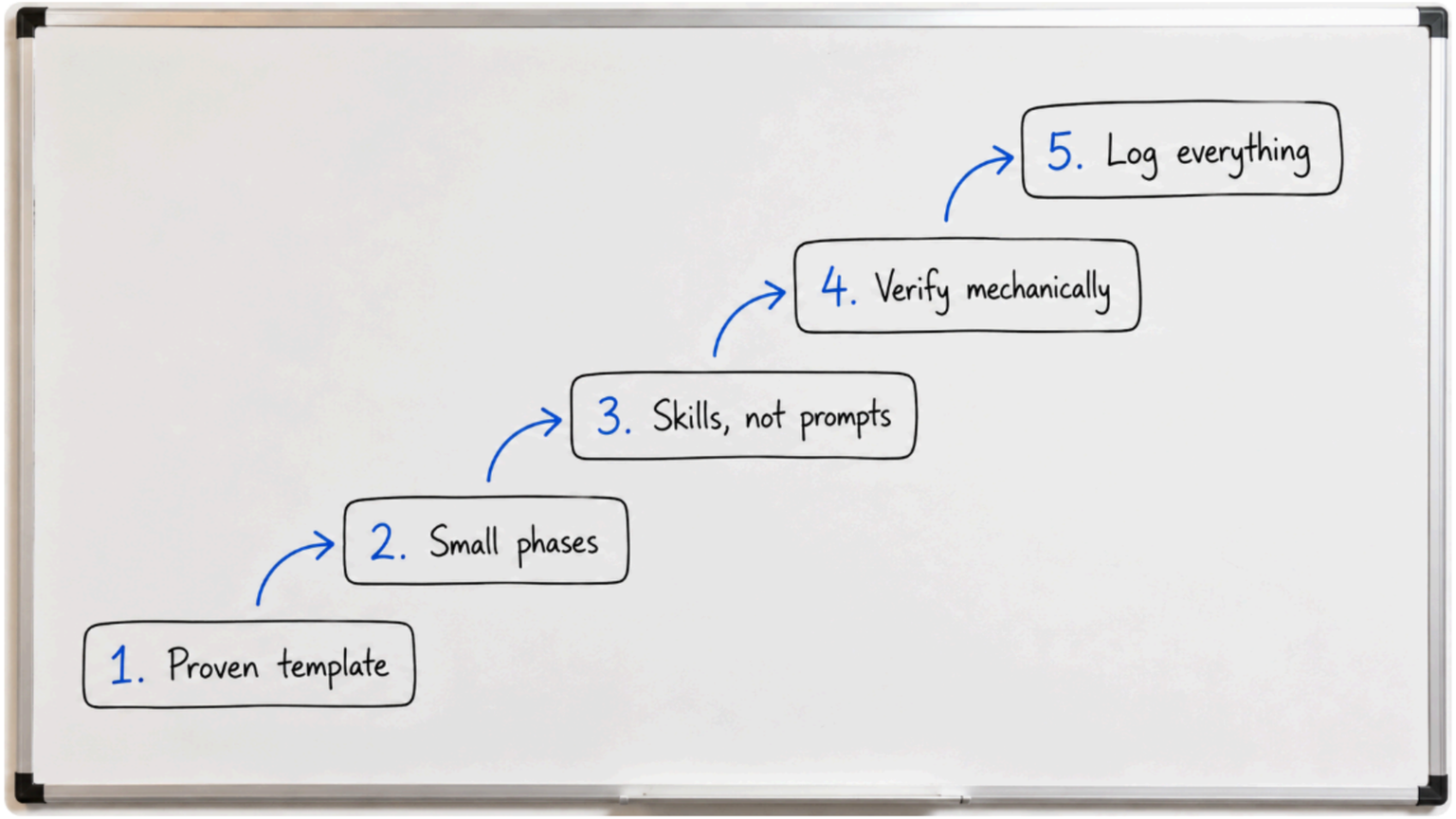
**$2–5**
model cost per full run

**22 → 22**
bugs discovered → guardrails added

**178**
commits · May 19 – Jul 12

TAKEAWAY · BEYOND TV

# Steal This Recipe for Your Domain



# Build the Harness Around Your Domain's Hard Parts

01

**Make the control flow deterministic.**

02

**Put domain judgment into skills and tools.**

03

**Treat verification and observability as product features.**

```
github.com/giolaq/tv-build-harness
```

```
explore: packages/harness
→ skills/  → packages/verification
```

# Thank You!

Giovanni Laquidara · Senior Developer Advocate, Amazon

`github.com/giolaq/tv-build-harness`

questions welcome — the emulator is warm